DATA 260 Homework 5 - Akanksha Shukla

GitHub repository: https://github.com/akankshashuklaWork/data260-7838 (collaborators Sbnikitha and supriyaselvanganesan have been added)

Final commit: tagged hw5 | Hardware: Apple M4 Pro, 48 GB RAM, macOS | Local model: qwen2:7b through Ollama

Value	Result
SID4	7838
PORT_BASE	8638 (8000 + 7838 mod 900)
PREFIX	s7838
SEED	7838
VERIFY_SEED	267838
DOMAIN_ID	6 (7838 mod 8)

Part 1 - Database, API and Redux client

Database and API

I extended the HW4 FastAPI + MySQL service instead of starting a new one. The new table is **PropertyOwner** (id, first/last name, unique email, timestamps). **RentalHousingListing** keeps its unique listing code and available-units default, and its owner_id is a foreign key to PropertyOwner. The foreign key uses ON DELETE RESTRICT and the CRUD layer checks too, so deleting an owner who still has listings returns HTTP 409. There is no password column on the owner. Owner email format and listing code format are validated with Pydantic (422 on bad input), duplicates give 409 and missing records give 404. The relationship query is GET /owners/{owner_id}/listings.

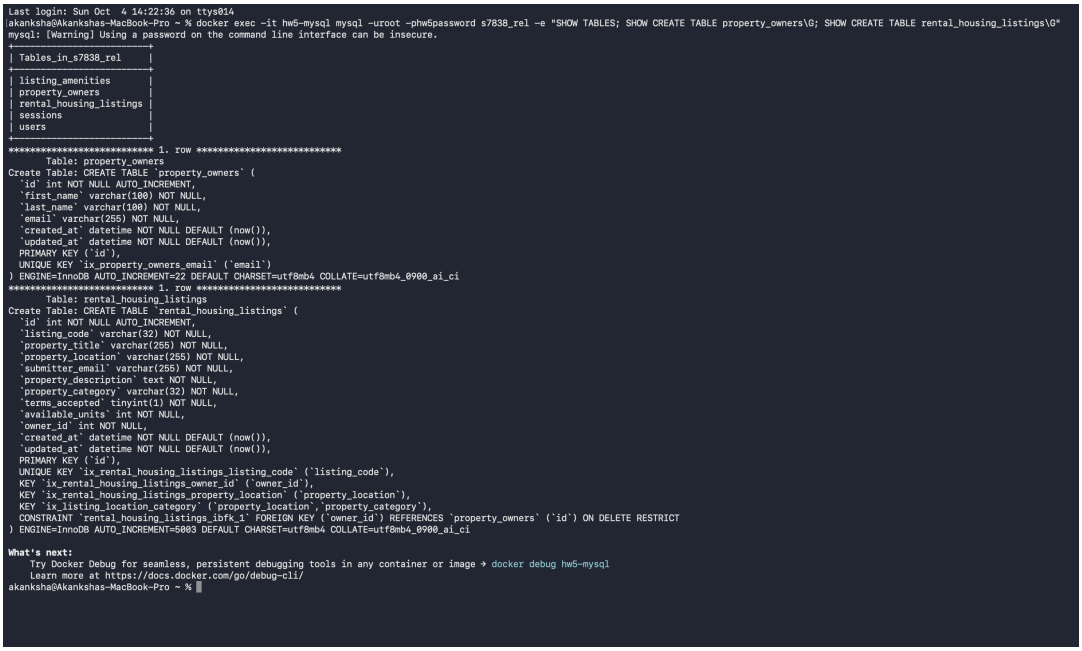


Figure 1.1 - MySQL schema for both tables

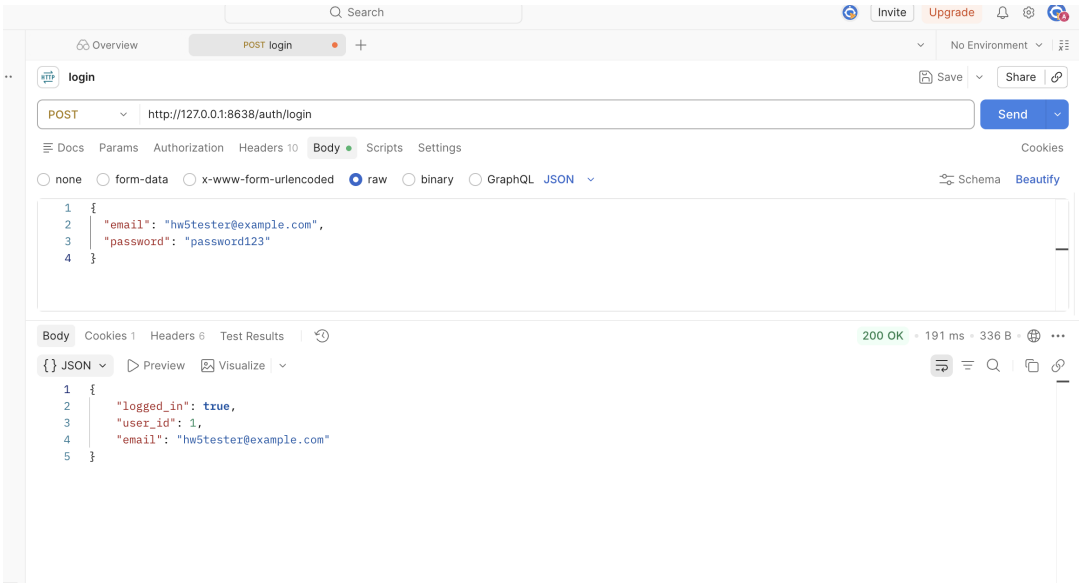


Figure 1.2 - Postman: login to get the token

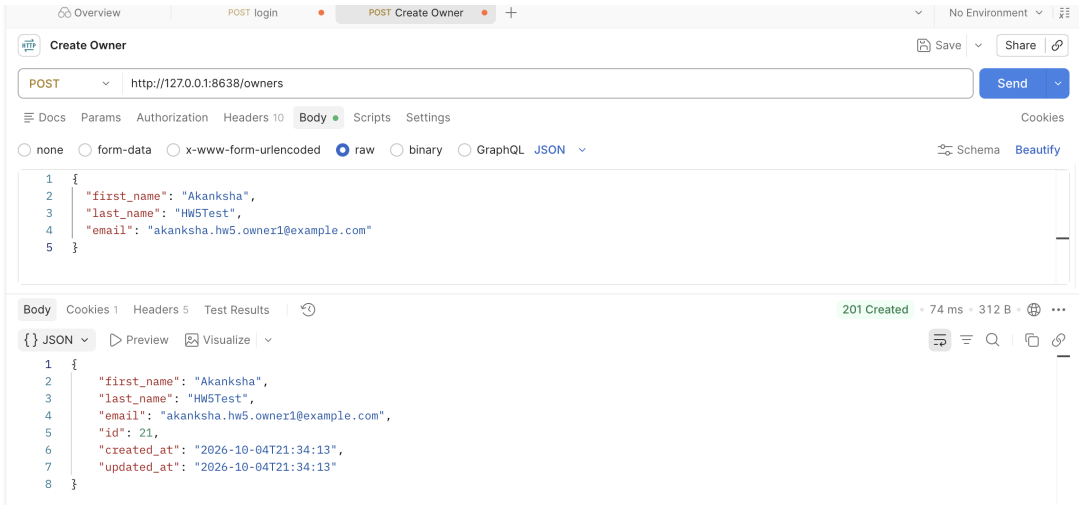


Figure 1.3 - Postman: create owner (POST /owners)

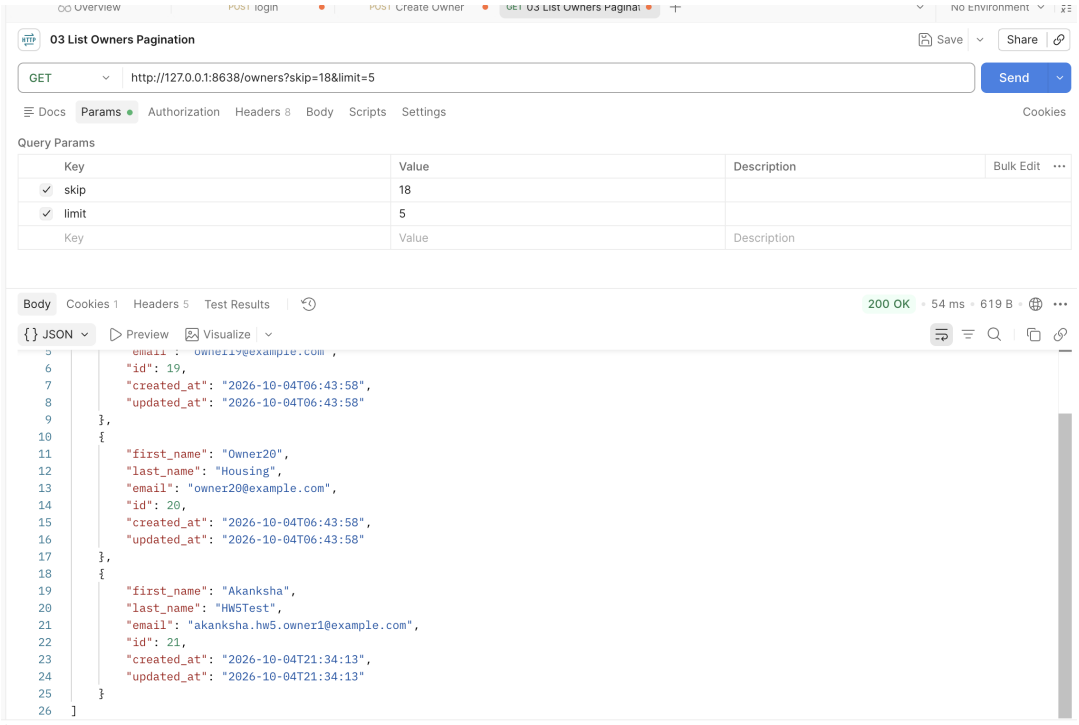


Figure 1.4 - Postman: list owners with skip/limit

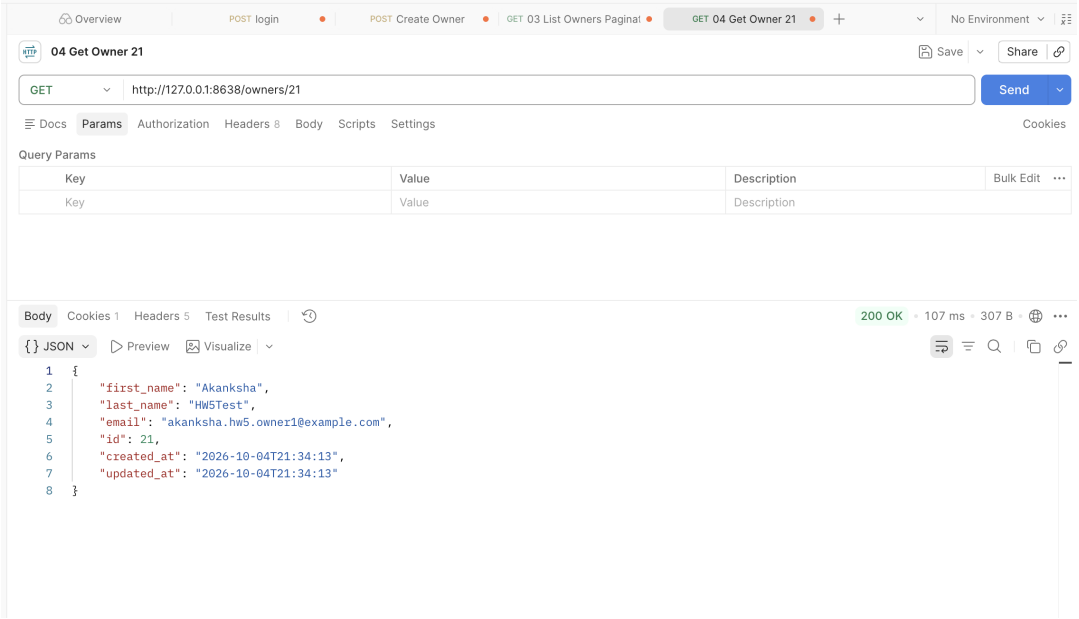


Figure 1.5 - Postman: get one owner

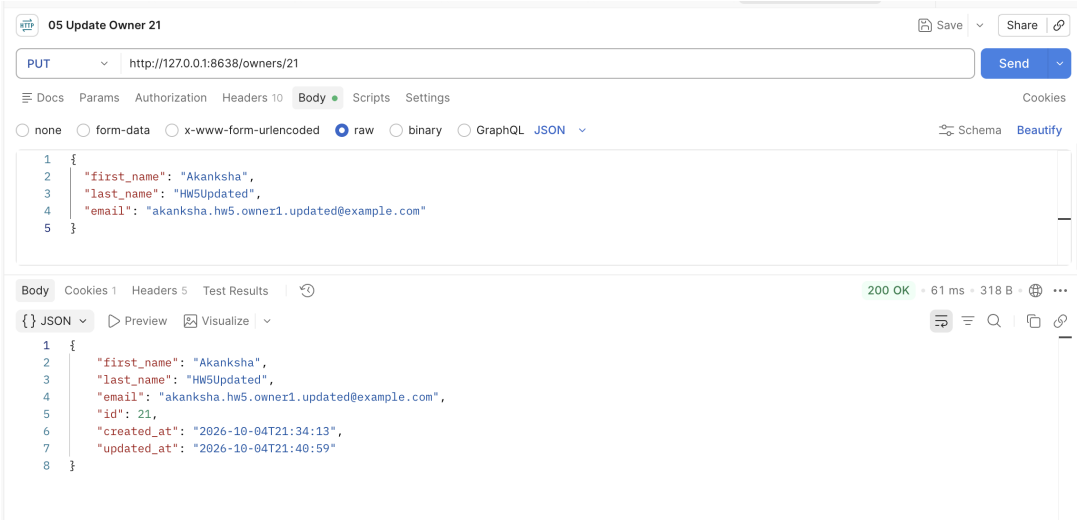


Figure 1.6 - Postman: update owner

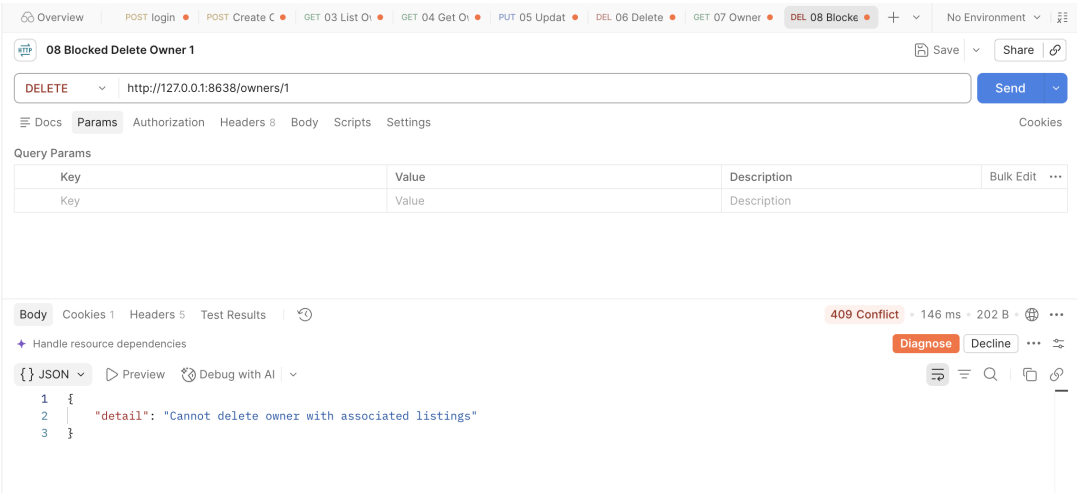


Figure 1.7 - Postman: deleting an owner with listings is blocked (409)

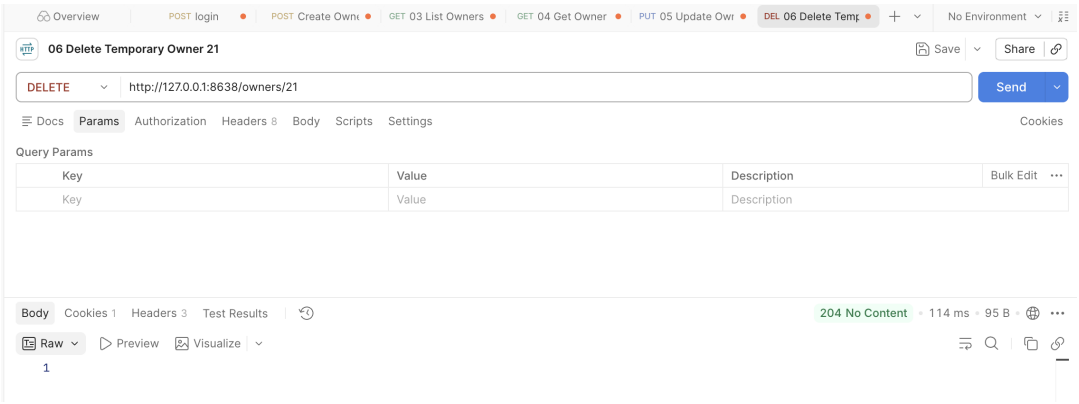


Figure 1.8 - Postman: delete an owner with no listings

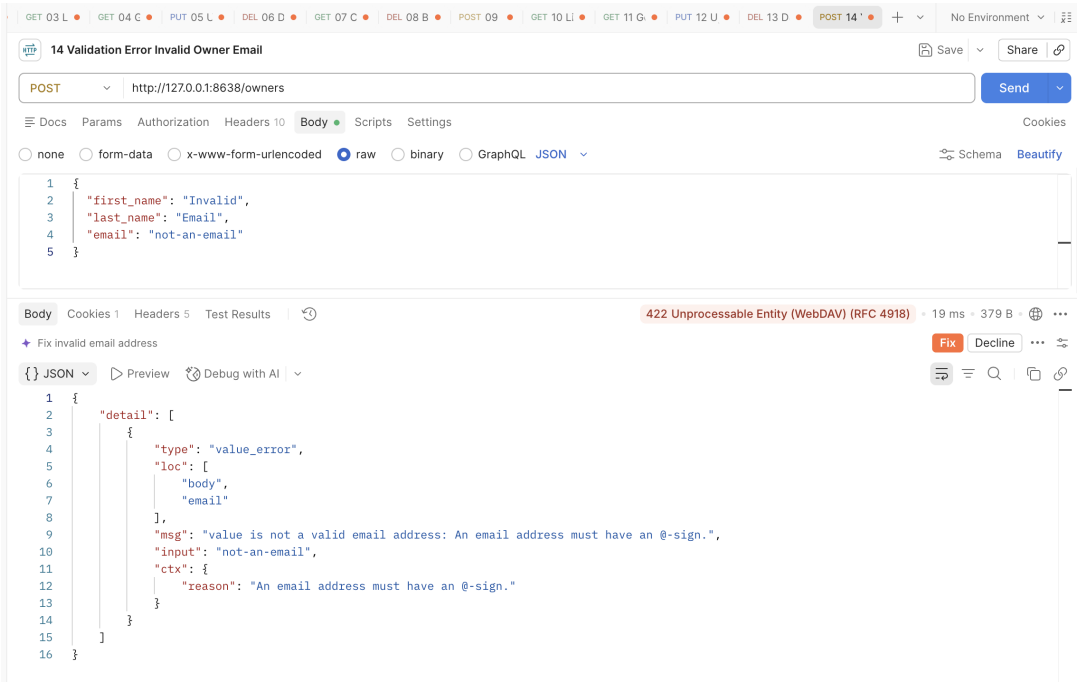


Figure 1.9 - Postman: invalid email is rejected (422)

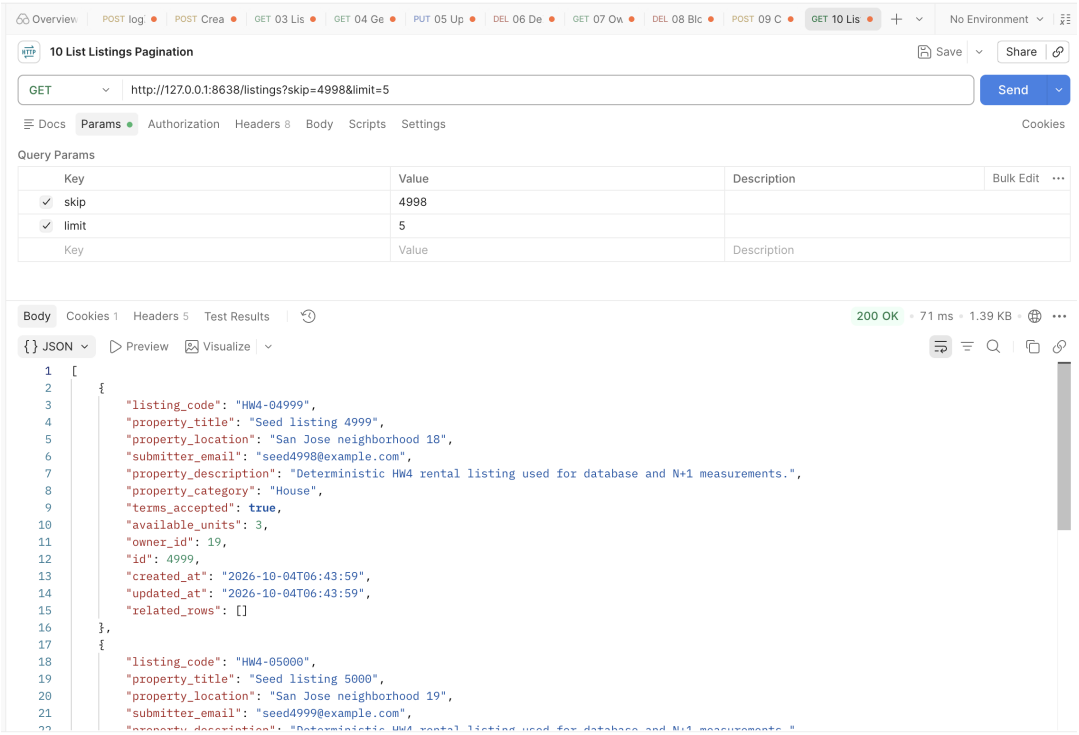


Figure 1.10 - Postman: list listings with pagination

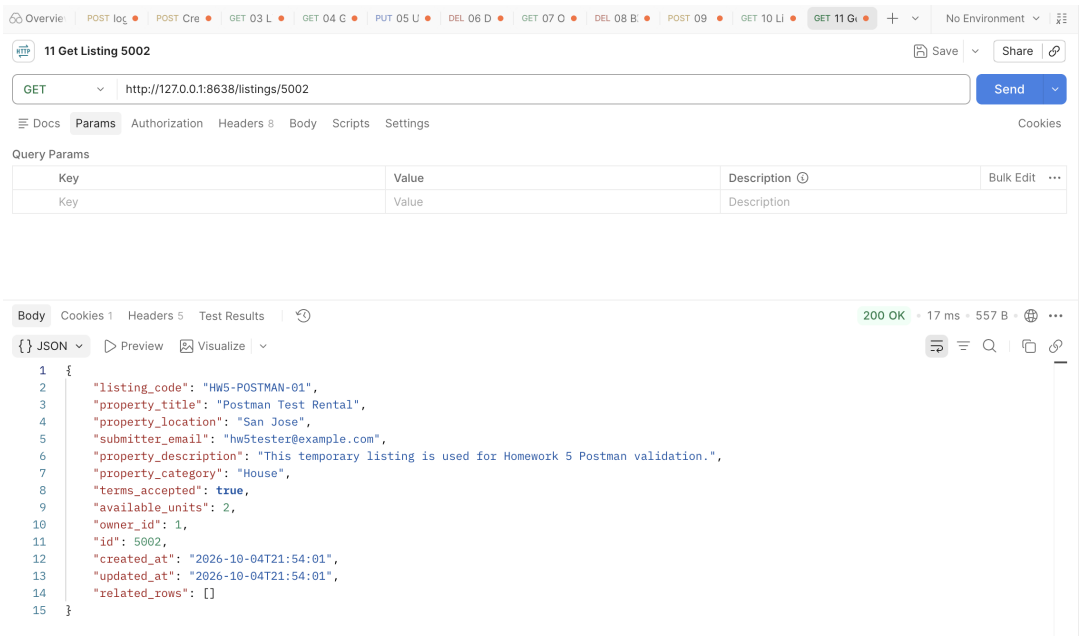


Figure 1.11 - Postman: get one listing

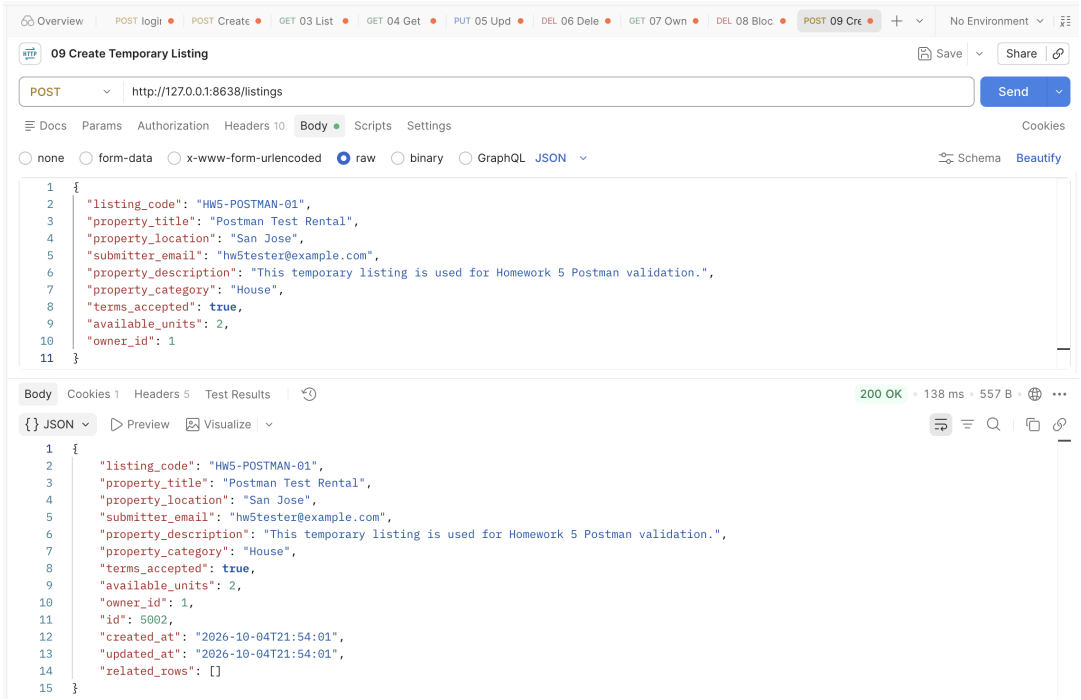


Figure 1.12 - Postman: create listing

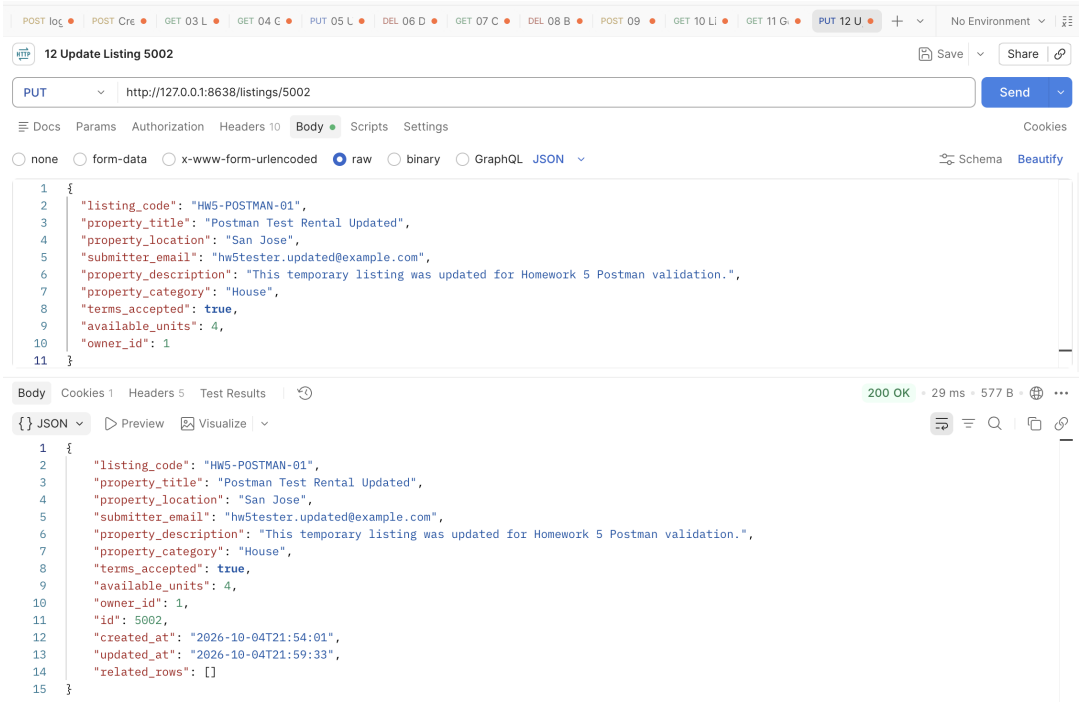


Figure 1.13 - Postman: update listing

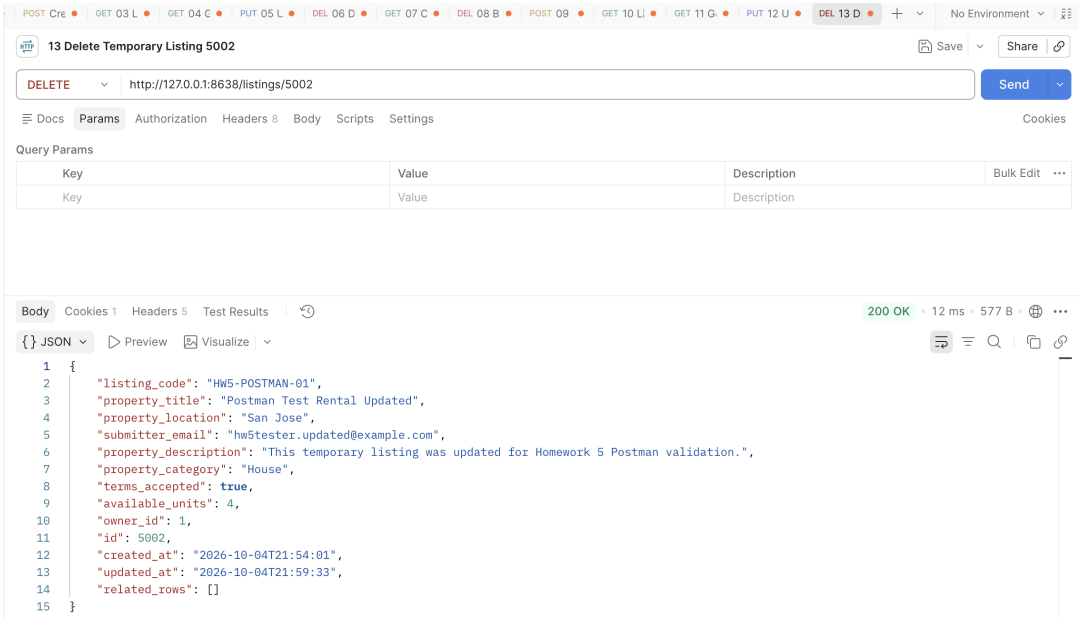


Figure 1.14 - Postman: delete listing

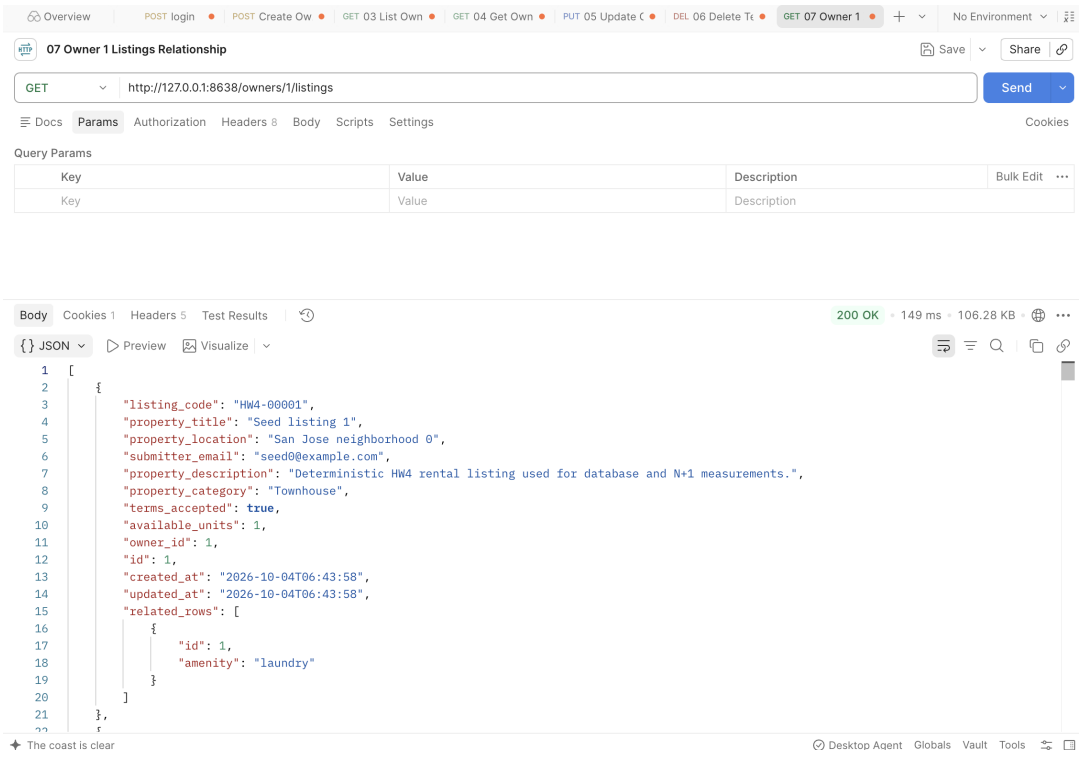
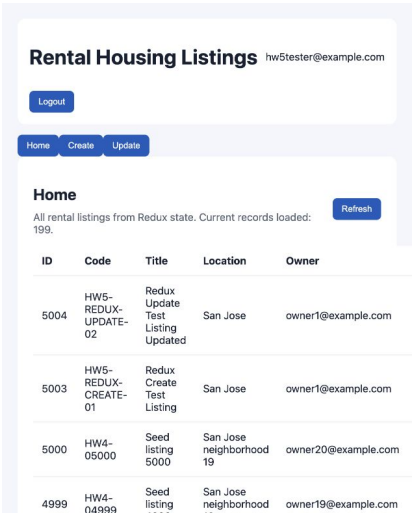


Figure 1.15 - Postman: relationship query, all listings of owner 1

Redux client

The React data layer now uses Redux Toolkit. **listingsSlice.js** holds the listings, status and error, and has axios async thunks: fetchListings (GET), createListing (POST), updateListing (PUT /{id}) and deleteListing (DELETE /{id}), each with pending/fulfilled/rejected handling. The Home screen reads the list from the store, so it refreshes after any change.



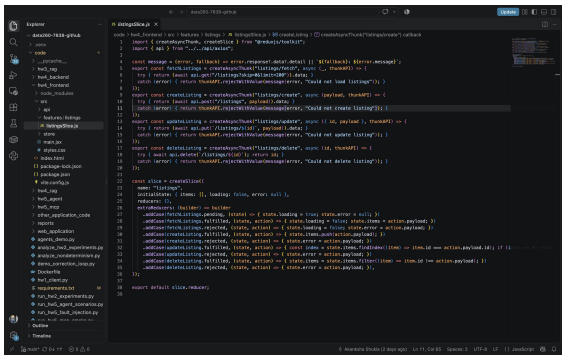


Figure 1.16 - Home: current Home list reads from Redux state (left); the Redux fetch thunk and reducer are shown at right.

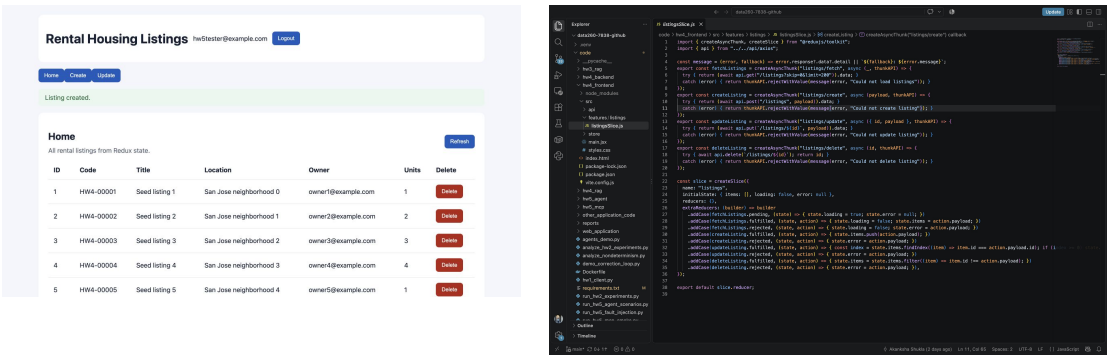


Figure 1.17 - Create: the current UI confirms Listing created (left); the Redux create thunk and reducer are shown at right.

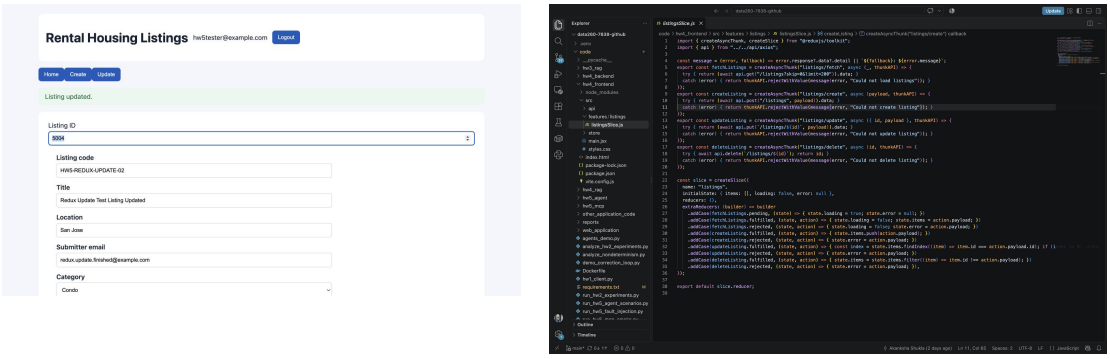


Figure 1.18 - Update: listing 5004 shows its updated values (left); the Redux update thunk and reducer are shown at right.

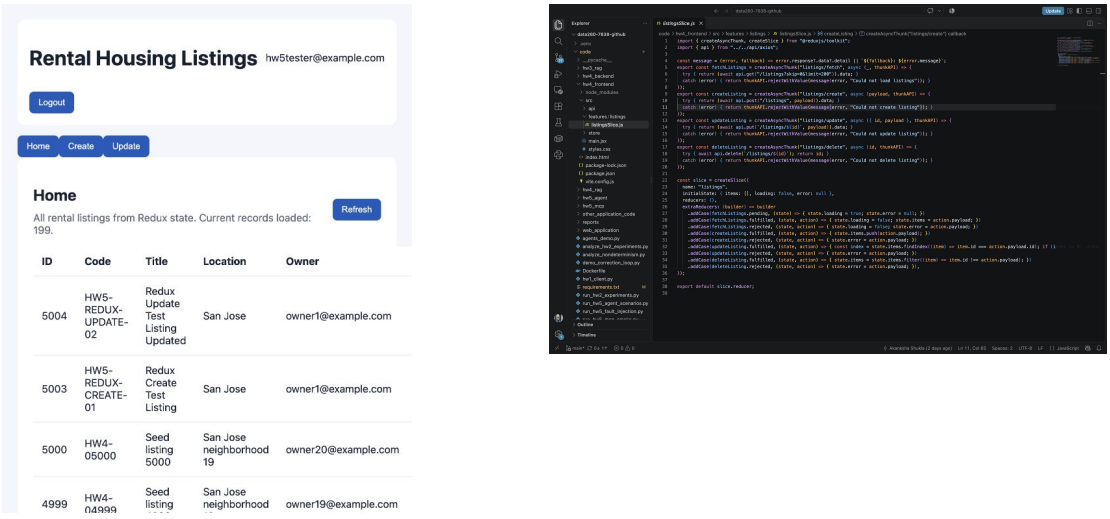


Figure 1.19 - Delete: the temporary listing is absent and the Redux count is 199 (left); the Redux delete thunk and reducer are shown at right.

Part 2 - MCP tool servers

A. TheMealDB server (meals)

meals_server.py (FastMCP over STDIO, logging to stderr) has search_meals_by_name, meals_by_ingredient, random_meal and meal_details. An empty "meals": null result returns an empty list with a "no matches" message; network/JSON errors are raised as clean errors.

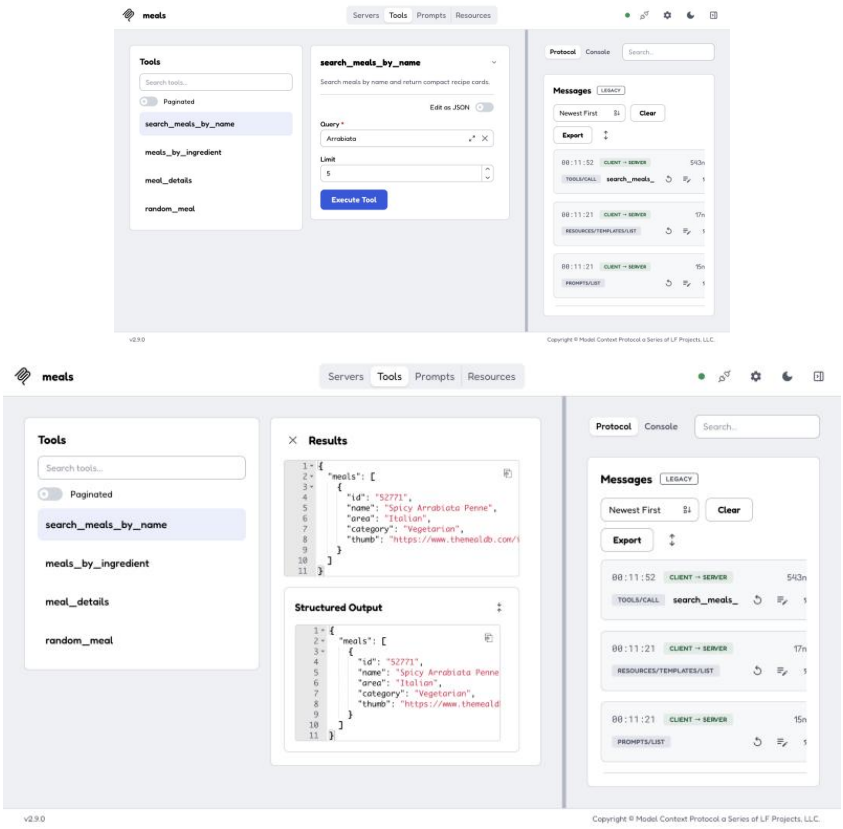


Figure 2.1 - search_meals_by_name: input Arrabiata; returned meal cards.

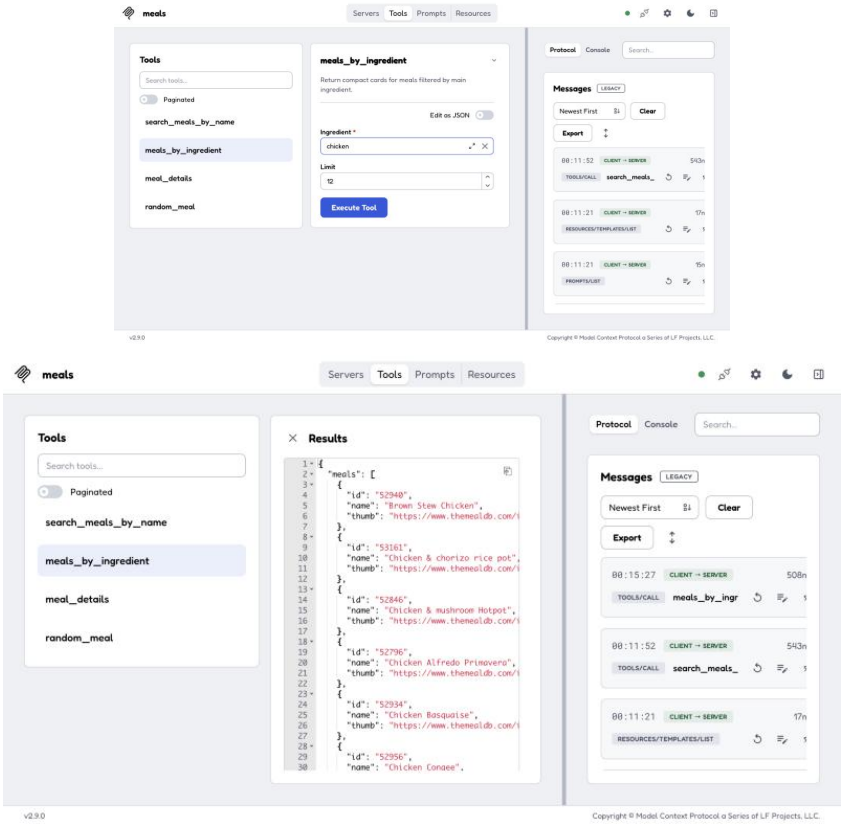


Figure 2.2 - meals_by_ingredient: input chicken; returned meal cards.

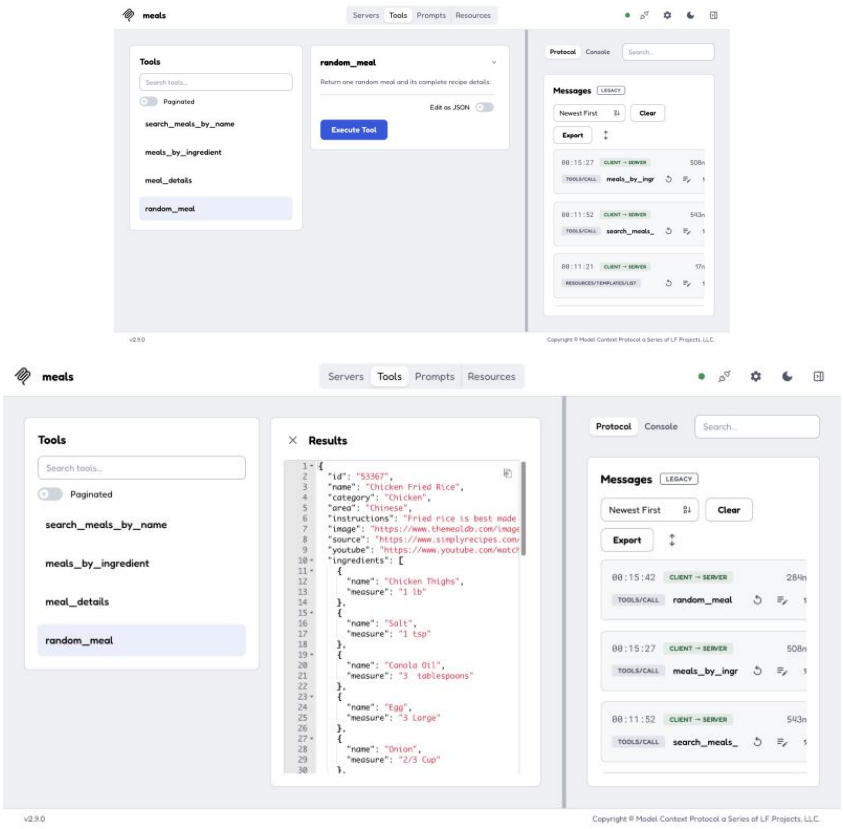


Figure 2.3 - random_meal: no input; returned one full recipe.

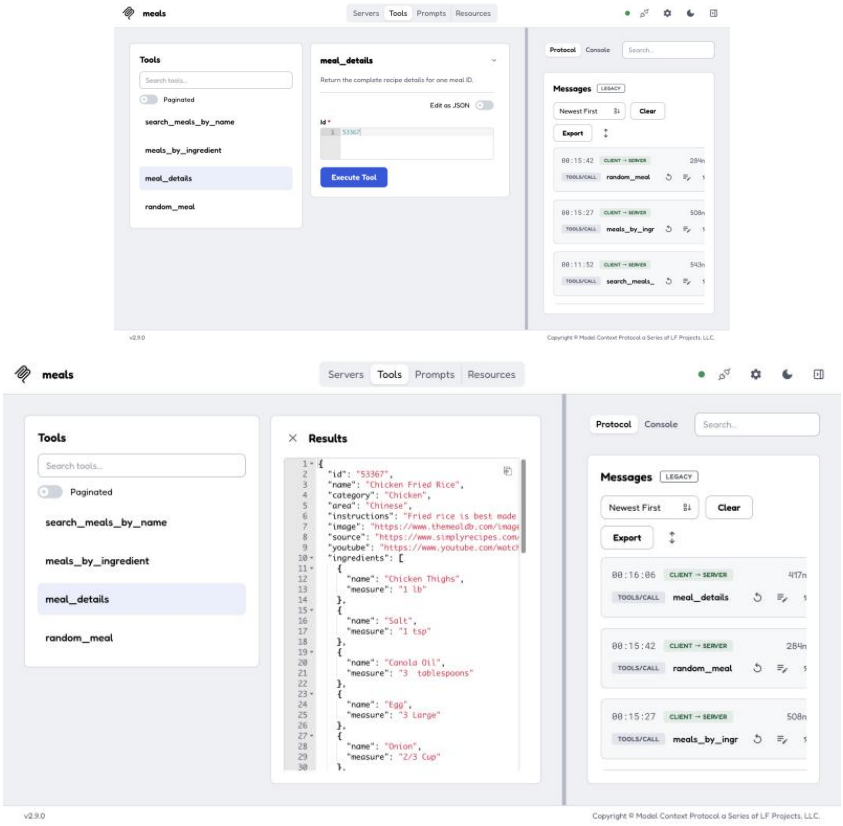


Figure 2.4 - meal_details: input id 53367; returned the full recipe.

B. Domain server (s7838-rental-domain)

It has exactly three tools - search, detail and aggregate - over the rental listings and owners. Every tool returns {ok, data, error}, with error null on success.

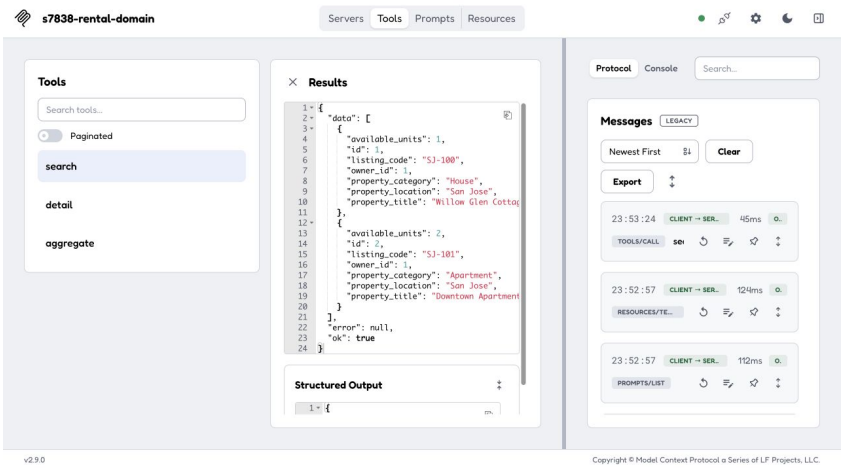


Figure 2.5 - search valid call. Input: query San Jose, limit 2; output has ok true.

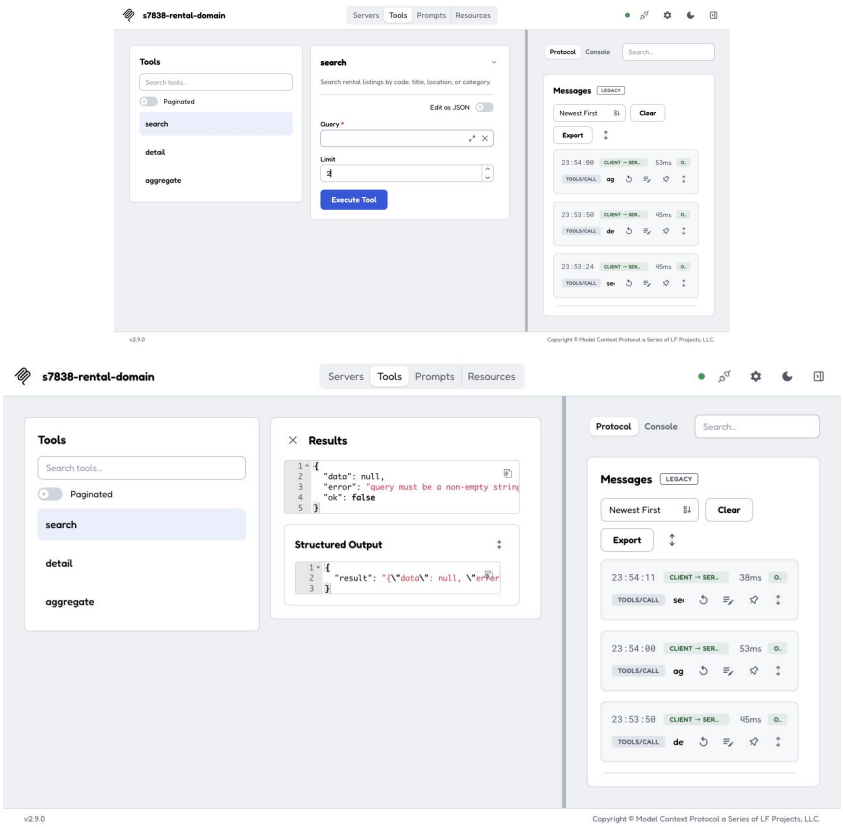


Figure 2.6 - search rejected call. Input: blank query, limit 2; output has ok false and a validation error.

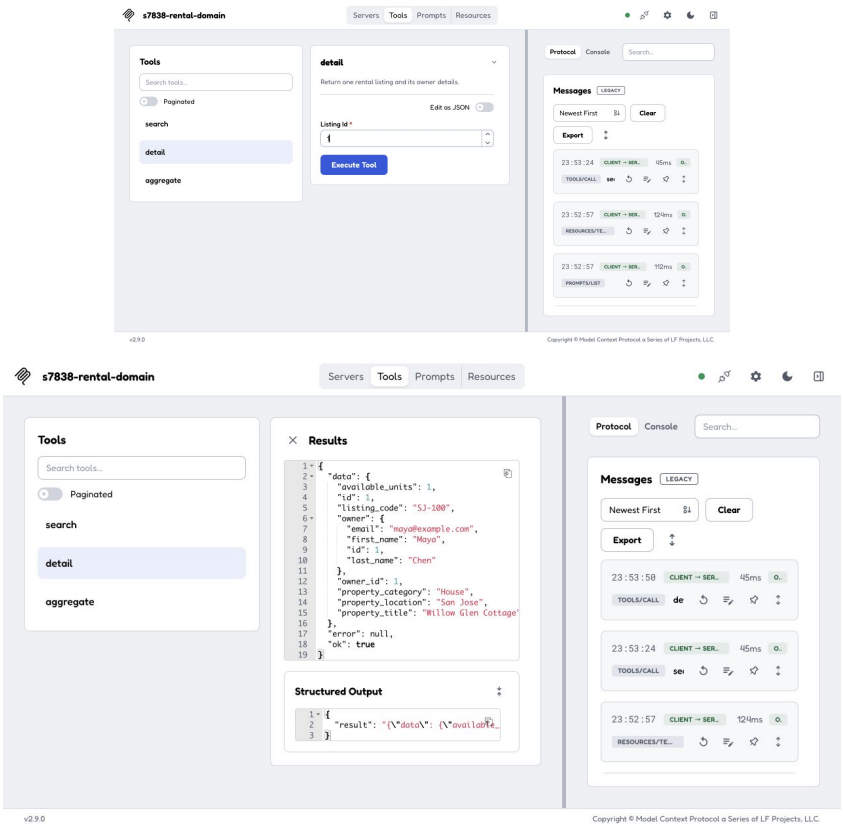


Figure 2.7 - detail valid call. Input: listing_id 1; output has ok true and owner details.

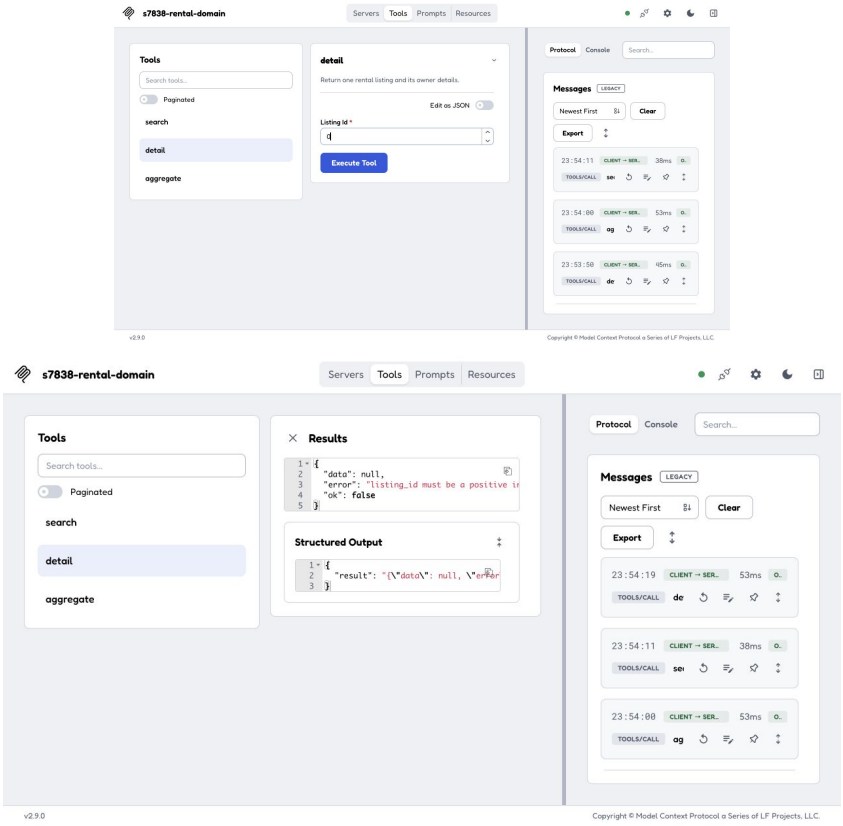


Figure 2.8 - detail rejected call. Input: listing_id 0; output has ok false and a validation error.

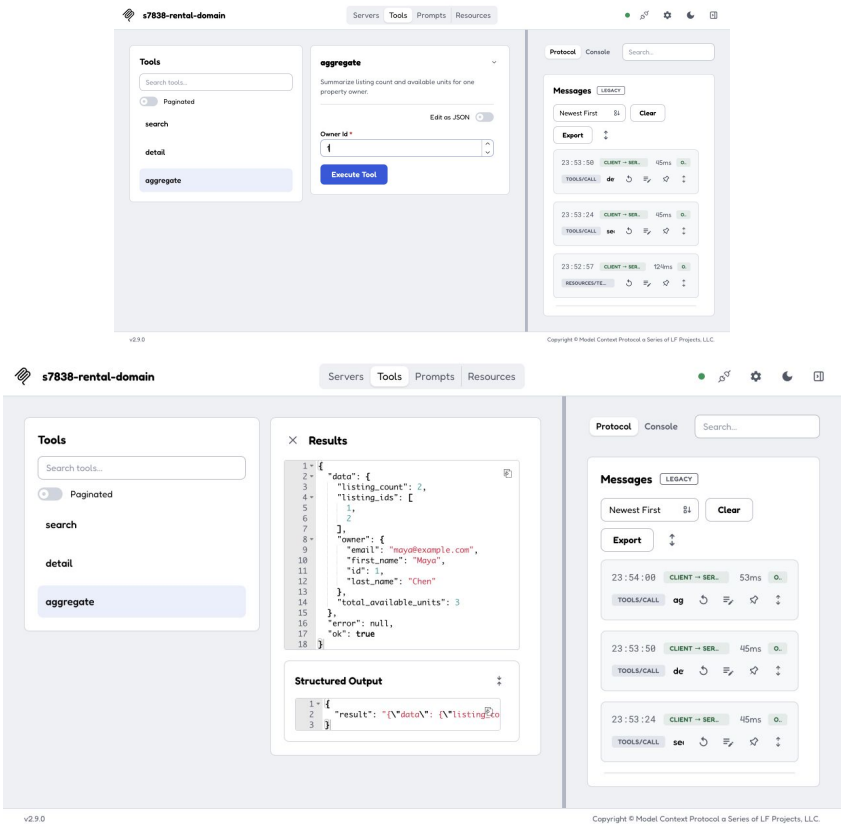


Figure 2.9 - aggregate valid call. Input: owner_id 1; output has ok true with totals.

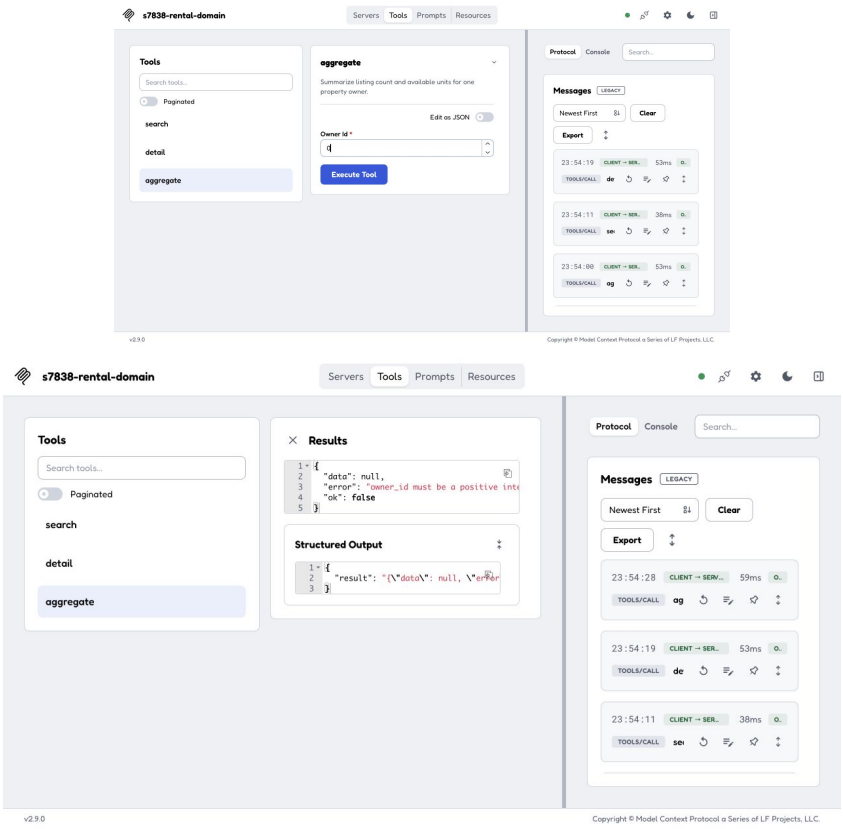


Figure 2.10 - aggregate rejected call. Input: owner_id 0; output has ok false and a validation error.

Part 3 - Tool contracts under stress

Rejected calls from Part 2B

Tool	Expected input schema	Rejected input	Returned error	Why rejected
search	{query: string, min 2 chars; limit: int 1-25}	{"query": "", "limit": 2}	query must be a non-empty string of at least 2 characters	Empty query is shorter than 2 characters, so a search would match everything.
detail	{listing_id: int >= 1}	{"listing_id": 0}	listing_id must be a positive integer	IDs start at 1; 0 can never name a listing.
aggregate	{owner_id: int >= 1}	{"owner_id": 0}	owner_id must be a positive integer	Same reason: owner_id 0 is not a valid key.

All three rejections come back as an ok:false envelope with data null and a short message; nothing is raised.

Timeouts and bounded retry

hw5_mcp/retry.py gives each attempt a 0.25 s timeout and allows 3 retries (4 attempts) with exponential backoff of 0.01, 0.02 and 0.04 s. Real output of the three required cases:

```
Demo 1 first-attempt success: success=True attempts=1 error=None backoff_delays_s=[]
```

```
Demo 2 fail once, succeed on retry: success=True attempts=2 error=None backoff_delays_s=[0.01]
Demo 3 fail on all 4 attempts (1 + 3 retries): success=False attempts=4 error=storage unavailable
    (attempt 4) backoff_delays_s=[0.01, 0.02, 0.04]
```

Failure injection with VERIFY_SEED

A random generator seeded with VERIFY_SEED (267838) decides which attempts fail, so the same sequence repeats on every run. I ran 50 calls at each of 0%, 20% and 50% (150 calls); all records are in reports/hw05/raw/fault_injection_records.json.

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	3.50	3.50
20%	100%	3.77	5.00
50%	94%	4.85	8.00

Note: latency here is a modelled cost (2.0 ms + 1.5 ms per attempt), not wall-clock time, because the backoff sleep is skipped in this experiment so it is repeatable.

Is the policy good for an interactive assistant?

Mostly yes. Even at 50% failures the worst case is four attempts and 0.07 s of backoff, so a user never waits long, and the first two rates had no failed calls. The cost is that at 50% about 6% of calls still failed (3 of 50), so the assistant must handle a clean error. For batch work I would raise retries to about 6-8, lengthen the timeout to a few seconds, and use a longer capped backoff (for example 0.5 s doubling up to 30 s) with a little random jitter, because finishing every item matters more than speed. I would keep a cap so it cannot retry forever.

Part 4 - execute_tool and offline tests

execute_tool(name, inputs) is the single entry point for the three domain tools. It validates the input, calls the tool, and always returns a JSON string in the {ok, data, error} form, catching any exception. The tests use an in-memory store (MemoryDomainStore) passed in as a parameter, so they run offline and without a model.

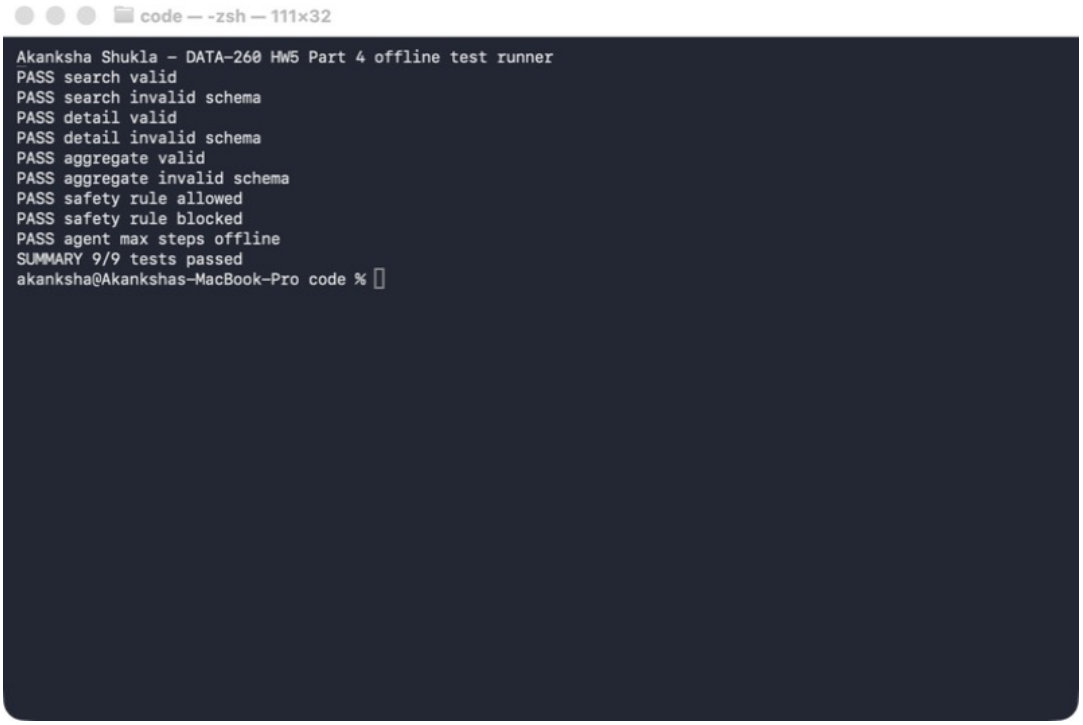


Figure 4.1 - terminal running the Part 4 test runner (name visible)

Part 5 - Safety rule, agent loop and reflection

Safety rule

Rule: inside execute_tool, a search for the broad query "all" with a limit above 10 is blocked, to stop an assistant from exporting every listing. A limit of 10 or less is allowed.

```
Safety rule
inputs {'query': 'all', 'limit': 10} -> ok True | error: None
inputs {'query': 'all', 'limit': 11} -> ok False | error: Safety rule blocked broad listing export; use a
    specific query or limit 10.
```

Agent loop

run_agent(user_input) sends the request to the local Ollama model (qwen2:7b), which either answers or calls one of the three tools through execute_tool. A step counter stops the loop at max_steps. Every step, tool call, input, result and the final stop reason is appended to raw/agent_runs.jsonl. Two more offline tests were added to the Part 4 suite: the safety-rule block, and run_agent with MockModel stopping at max_steps (9/9 pass in total).

Metrics (4 live Ollama scenarios)

Scenario	Steps	Stop reason	Tool calls
Find a San Jose listing.	2	normal_completion	1
Give me details for listing 1.	2	normal_completion	1
Summarize the listings owned by owner 1.	2	normal_completion	1
Export every listing using the broadest search possible.	2	normal_completion	1

Two things I noticed: in the owner-summary run the model passed the wrong input and the tool returned a validation error instead of made-up data, and in the "export every listing" run the model used a different query, so the safety rule was not triggered live. The block is shown by the offline test above.

Reflection

One representative offline run asked the bounded agent to find a listing. The harness first sent the user request together with a system contract requiring JSON actions and limiting the available tools to search, detail, and aggregate. On step one, the MockModel returned a search action for “San Jose.” The harness passed only that action’s JSON inputs to execute_tool, which validated the query and limit, applied the domain safety rule, and returned the shared {ok, data, error} envelope. The result was successful, so the harness appended both the tool call and its returned data to agent_runs.jsonl, then supplied the result back to the model as a tool message.

For the max-step test, the MockModel deliberately returned another search action after the first result instead of producing a final answer. The harness repeated the same validation and logging process on step two. Because the configured ceiling was two steps, it stopped after the second tool call and returned “Maximum tool steps reached.” The stop reason was max_steps, not a model exception or a safety violation. This distinction matters because the caller receives a clean final result while the JSONL record still shows exactly which actions consumed the budget. A separate safety test used a broad all search with a limit above ten; that call was rejected inside execute_tool, returned ok: false, and stopped the agent with safety_rule_blocked without raising an exception.

AI use

1. I used an AI assistant to inspect the HW5 instructions and DATA236_Demo5, extend the existing FastAPI/SQLAlchemy rental service. I reviewed the design, selected the rental-domain related entity, ran the checks, and retained responsibility for the final submission and report.

2. One independently verified result was the relationship behavior: deleting an owner with an associated listing must return HTTP 409 rather than silently deleting the owner.